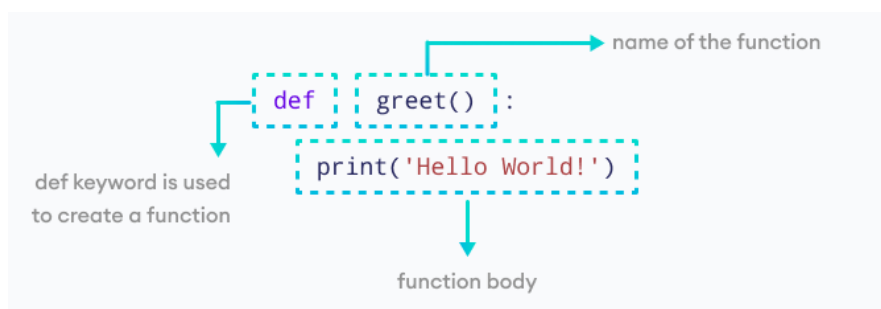


Python Functions

- A function is a block of code which only runs when it is called.
- You can pass data, known as parameters, into a function.
- A function can return data as a result.

1. Creating a Function

In Python a function is defined using the **def** keyword:



2. Calling a Function

To call a function, use the function name followed by parenthesis:

```
def my_function():  
    print("Hello from a function")
```

```
my_function()
```

```
Hello from a function
```

3. Arguments

Information can be passed into functions as arguments.

Arguments are specified after the function name, **inside the parentheses**. You can add as many arguments as you want, just **separate them** with a **comma**

The following example has a function with one argument (name). When the function is called, we pass along a first name, which is used inside the function to print the full name:

```
def my_function(name):  
    print(name + " Gunasekera")
```

```
my_function("Amal")  
my_function("Kamal")  
my_function("Sunil")
```

Output:

```
Amal Gunasekera  
Kamal Gunasekera  
Sunil Gunasekera
```

3.1 Parameters or Arguments

The terms **parameter** and **argument** can be used for the same thing: information that are passed into a function.

From a function's perspective:

A parameter is the variable listed inside the parentheses in the function definition.

An argument is the value that is sent to the function when it is called.

3.2 Number of Arguments

By default, a function must be called with the correct number of arguments. Meaning that if your function **expects 2 arguments**, you have to **call the function with 2 arguments**, not more, and not less.

```
def my_function(fname, lname):  
    print(fname + " " + lname)
```

```
my_function("Sunil", "Gunasekera")
```

Output:

```
Sunil Gunasekera
```

If you try to call the function with 1 or 3 arguments, you will get an error:

```
def my_function(fname, lname):  
    print(fname + " " + lname)
```

```
my_function("Sunil")
```

```
Traceback (most recent call last):  
  File "./prog.py", line 4, in <module>  
TypeError: my_function() missing 1 required positional argument:
```

3.4 Arbitrary Arguments, *args

If you do not know how many arguments that will be passed into your function, add **a *** before the parameter name in the function definition.

This way the function will receive a *tuple* of arguments, and can access the items accordingly:

If the number of arguments is unknown, add a ***** before the parameter name:

```
def my_function(*names):  
    print("The youngest child is " + names[2])  
  
my_function("Kasun", "Pramodh", "Nisal")
```

Output:

```
The youngest child is Nisal
```

3.5 Keyword Arguments

- You can also send arguments with the **key = value** syntax.
- This way the order of the arguments does not matter.

```
def my_function(child3, child2, child1):  
    print("The youngest child is " + child3)  
  
my_function(child1 = "Kasun", child2 = "Pramodh", child3 = "Nuwan")
```

Output:

```
The youngest child is Nuwan
```

5. Default Parameter Values

The following example shows how to use a default parameter value.

If we call the function without argument, it uses the **default value**:

```
def my_function(country = "Sri Lanka"):
    print("I am from " + country)
```

```
my_function("Sweden")
my_function("India")
my_function()
my_function("Brazil")
```

Output:

```
I am from Sweden
I am from India
I am from Sri Lanka
I am from Brazil
```

6. Passing a List as an Argument

You can send any data types of argument to a function (string, number, list, dictionary etc.), and it will be treated as the same data type inside the function.

E.g. if you send a List as an argument, it will still be a List when it reaches the function:

```
def my_function(foodList):
    for x in foodList:
        print(x)
```

```
fruits = ["apple", "banana", "cherry"]
```

```
my_function(fruits)
```

Output:

```
apple
banana
cherry
```

7. Return Values

To let a function return a value, use the **return** statement:

```
def my_function(x):  
    return 5 * x
```

Output:

```
print(my_function(3))  
print(my_function(5))  
print(my_function(9))
```

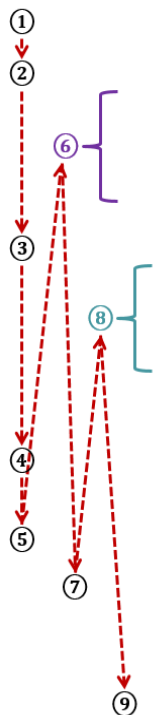
```
15  
25  
45
```

We can get the same output,

```
def my_function(x):  
    value = print(5 * x)  
  
    return value
```

```
my_function(3)  
my_function(5)  
my_function(9)
```

Example:



```
import random  
  
def check_credentials(username, password):  
    if username == 'admin' and password == 'abc123':  
        return True  
    else:  
        return False  
  
def generate_randoms():  
    number = random.randint(1, 100)  
    letter = random.choice('ABCDEFGHIJKLMNOPQRSTUVWXYZ')  
    print('Your number is', number)  
    print('Your letter is', letter)  
  
print('Please Log In.')  
uname = input('Username: ')  
pword = input('Password: ')  
  
if check_credentials(uname, pword):  
    print('Login Successful.')  
    generate_randoms()  
else:  
    print('Invalid credentials.')
```

Python

```
Please Log In.  
Username: admin  
Password: abc123  
Login Successful.  
Your number is 45  
Your letter is S
```

- ① Import the "random" module
 - ② Define (but don't execute) the check_credentials function
 - ③ Define (but don't execute) the generate_randoms function
 - ④ Print the login message and prompt the user to input a username and password
 - ⑤ Call the check_credentials() function to determine the result of the if statement. The username and password inputted by the user are passed as parameters/arguments to check_credentials()
 - ⑥ The code in check_credentials() runs, returning True since the username was "admin" and the password was "abc123"
 - ⑦ Print the login success message and call the generate_randoms() function
 - ⑧ The code in generate_randoms() runs, generating and printing a random number and letter
 - ⑨ Control returns back to the main program, which ends since there are no more statements
- Also note that it is perfectly fine for the code of a function to call another function.

8. Python Scope

A variable is only available from inside the region it is created. This is called **scope**.

8.1 Local Scope

A variable created inside a function belongs to the *local scope* of that function, and can only be used inside that function.

A variable created inside a function is available inside that function:

```
def myfunc():  
    x = 300  
    print(x)  
  
myfunc()
```

Function Inside Function

As explained in the example above, the variable `x` is not available outside the function, but it is available for any function inside the function:

The local variable can be accessed from a function within the function:

```
def myfunc():  
    x = 300  
    def myinnerfunc():  
        print(x)  
    myinnerfunc()
```

```
myfunc()
```

8.2 Global Scope

A variable created in the main body of the Python code is a global variable and belongs to the global scope.

Global variables are available from within any scope, global and local.

A variable created outside of a function is global and can be used by anyone:

```
x = 300
```

```
def myfunc():  
    print(x)
```

```
myfunc()
```

```
print(x)
```

Output:

300

300

Example:

```
def example():  
    print('var in funct:', var)
```

```
var = 'A'  
example() # var is NOT passed in as a parameter  
print('var in global:', var)
```

Python

var in funct: A
var in global: A

8.3 Naming Variables

If you operate with the same variable name inside and outside of a function, Python will treat them as two separate variables, one available in the global scope (outside the function) and one available in the local scope (inside the function):

The function will print the local `x`, and then the code will print the global `x`:

```
x = 300
```

```
def myfunc():  
    x = 200  
    print(x)
```

Output:

```
myfunc()
```

```
print(x)
```

200

300

Example:

```
def example():  
    var = 'B'  
    print('var in funct:', var)  
  
var = 'A'  
example()  
print('var in global:', var)
```

Python

var in funct: B
var in global: A

```
def example():  
    global var  
    var = 'B'  
    print('var in funct:', var)  
  
var = 'A'  
example()  
print('var in global:', var)
```

Python

var in funct: B
var in global: B

8.4 Global Keyword

If you need to create a global variable, but are stuck in the local scope, you can use the `global` keyword.

The `global` keyword makes the variable global.

If you use the `global` keyword, the variable belongs to the global scope:


```
def myfunc():  
    global x  
    x = 300  
  
myfunc()  
  
print(x)
```

Also, use the **global** keyword if you want to make a change to a global variable inside a function.

To change the value of a **global** variable inside a function, refer to the variable by using the global keyword:

```
x = 300
```

```
def myfunc():  
    global x  
    x = 200
```

Output:

```
myfunc()
```

```
200
```

```
print(x)
```

8.5 Parameters

When you define a function, you can define its parameters

- This defines what data can be *passed into the function* when it is called in your code.
- Hence, a function can be instructed to perform its task *on the data you give it*, or *in a particular way* (or both).
- If a function does not need any data/instructions, it may have no parameters (rare).

function can have **parameters defined**, and when it is called you specify the arguments that are passed into the function. i.e., the **parameters are the placeholders** in the function that contain **the arguments when it is called**.

Example 1:

```
def repeat(text, count = 2):  
    result = ''  
    for num in range(count):  
        result = result + text  
    return result  
  
print(repeat('Hello!', 4))  
  
word = 'Duck, '  
phrase = repeat(word) + 'Goose!'  
print(phrase)
```

Python

```
Hello!Hello!Hello!Hello!  
Duck, Duck, Goose!
```

Example 2:

```
def dec_to_percent(num):  
    if num > 1:  
        num = '100%'  
    elif num < 0:  
        num = '0%'  
    else:  
        num = str(round(num * 100)) + '%'  
    return num  
  
num = 0.235  
print('Result of dec_to_percent(num):', dec_to_percent(num))  
print('Value of 'num' in main program:', num)
```

Python

Assigning a value to `num` in the function does *not* change `num` in the main program – they are different variables

```
Result of dec_to_percent(num): 24%  
Value of 'num' in main program: 0.235
```

9. Returning Data from Functions

Just as parameters allow us to pass data *into* a function, a return statement allows us to return data *from* a function back to the code that called it

- This allows functions to remain independent of the code that calls them, passing data in and returning the result as needed.

A return statement is quite simple: `return <value>`

- The value can be of any data type/structure (string, int, list...), and can be a literal value, a value referred to by a variable, a function that will return a value...

`return 'HD'`

`return result`

`return round(result, 2)`

```
def add(num_one, num_two):  
    result = num_one + num_two  
    return result
```

Python

```
def add(num_one, num_two):  
    return num_one + num_two
```

Python

Same Output

This version is more efficient!